

초급 백엔드 스터디

6주차 - 컨트롤러 계층

지난 주에는...

- 서비스 계층 작성
- 서비스 계층 단위 테스트 작성

이번 주에는...

- 컨트롤러 계층 작성
- Postman을 사용한 API 테스트

스프링 Layered Architecture

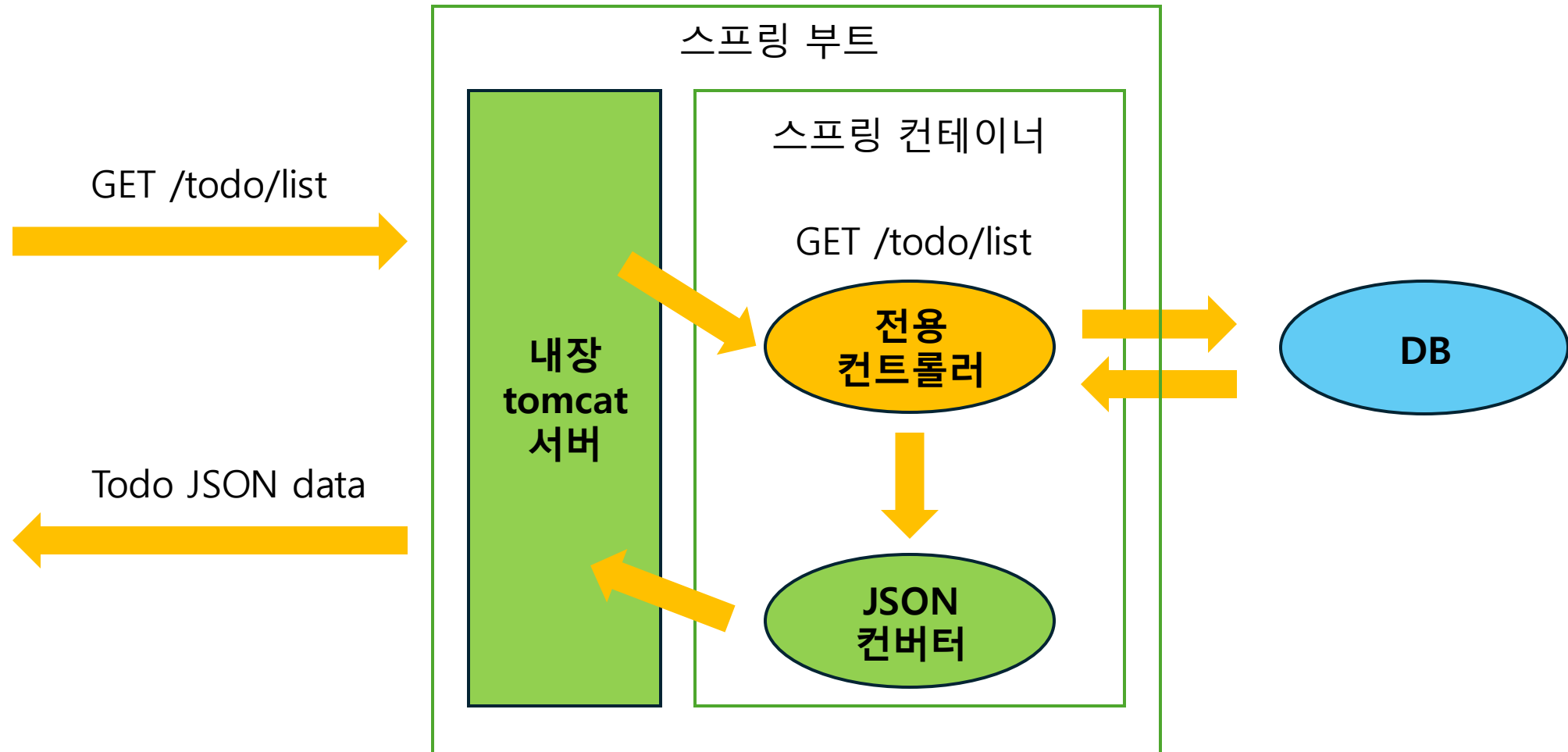


스프링 Layered Architecture

컨트롤러

- 클라이언트의 요청을 받고, 응답을 보내는 계층
- **DTO** (Data Transfer Object)를 사용하여
서비스 계층과 데이터를 주고받는다.

스프링 복습



컨트롤러 계층



컨트롤러 계층

Todo 컨트롤러 클래스

GET
/todo/list
처리 메서드

POST
/todo
처리 메서드

DELETE
/todo/{id}
처리 메서드

컨트롤러 계층

- todo 패키지 아래에 TodoController를 생성한다.



컨트롤러 계층

- 컨트롤러도 하나의 객체만 생성하면 되므로 빈으로 등록한다.
- **@Controller**를 사용하면 컴포넌트 스캔의 대상이 된다.

```
@Controller
public class TodoController {

}
```

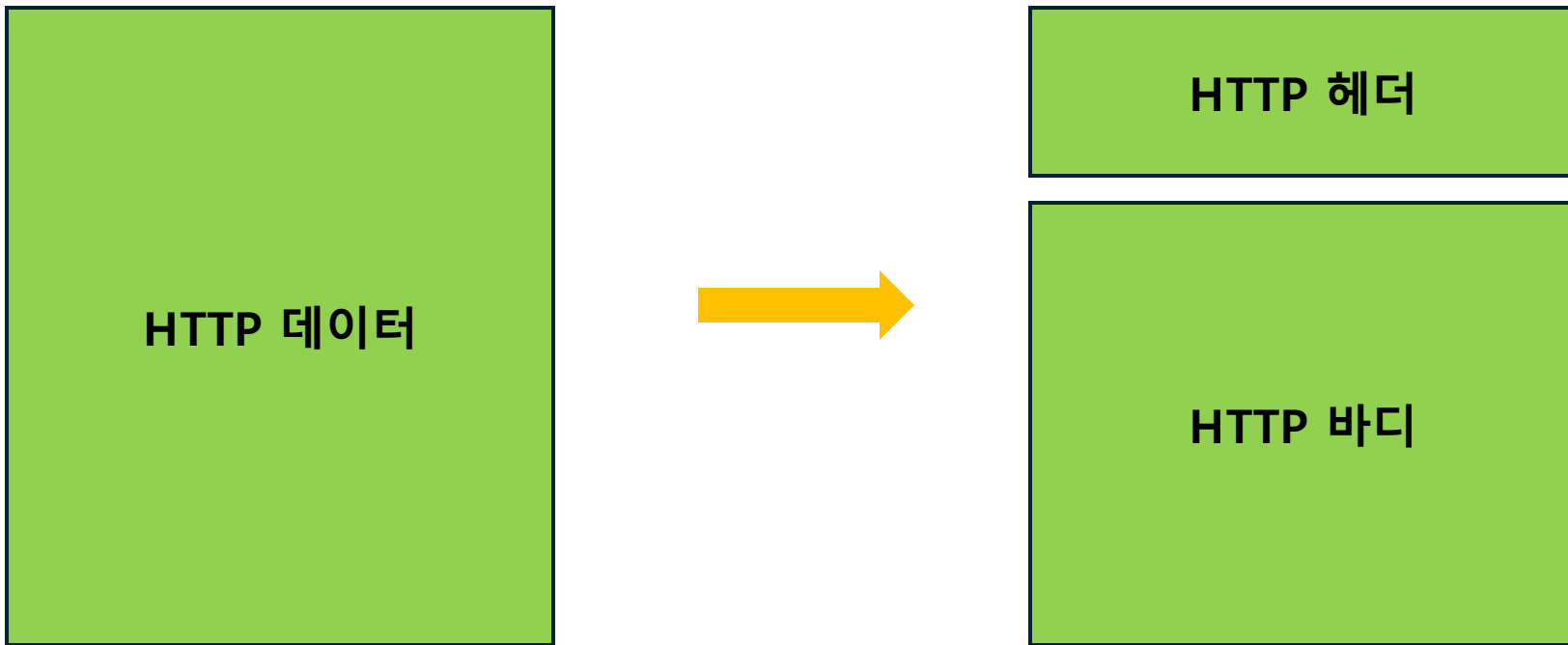
```
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Component
public @interface Controller {

    /**
     * Alias for Component.value.
     */
    @AliasFor(annotation = Component.class)
    String value() default "";

}
```

프로토콜과 HTTP - 복습

- HTTP로 주고받는 데이터의 구조



프로토콜과 HTTP - 복습

- 보통 어플리케이션과 관련된 데이터는 **body**에 담는다.
(header에 담는 경우도 있는데, 이번 스터디에서는 다루지 않습니다.)
- HTTP 요청으로 보내는 데이터는 **Request Body**
- HTTP 응답으로 보내는 데이터는 **Response Body** 에 담긴다.

컨트롤러 계층

- API 서버는 **json 데이터를 응답**하는 경우가 많다.
- **@ResponseBody** 사용하면 메서드가 자바 객체를 반환했을 때, 객체를 **json 데이터로 변환**해서 response body에 담아 응답한다.

```
@Controller
@ResponseBody
public class TodoController {

}
```

컨트롤러 계층

- 보통은 편의를 위해 **@RestController**을 대신 사용한다.

```
@RestController
public class TodoController {

}
```

```
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Controller
@ResponseBody
public @interface RestController {

    |

    The value may indicate a suggestion for a logical component name
    in case of an autodetected component.

    Returns: the suggested component name, if any (or empty String)

    Since: 4.0.1
```

컨트롤러 계층

- 컨트롤러 계층은 서비스 계층에 의존한다.
- **생성자 주입** 방식으로 서비스 빈을 주입 받는다.

```
@RestController
@RequiredArgsConstructor
public class TodoController {

    private final TodoService todoService;

}
```

컨트롤러 계층

- **@RequestMapping**을 이용하여
메서드가 처리할 요청 method, url을 지정한다.

```
@RequestMapping(method = RequestMethod.POST, value = "/todo")  
public void createTodo() throws Exception {  
    todoService.createTodo(content: "content", userId: 1L);  
}
```



POST /todo로 요청을 받으면
createTodo() 메서드 호출

컨트롤러 계층

- TodoController는 모든 todo 관련 요청을 처리하는 클래스
- 이때 API 설계에서 todo 관련 요청은 모두 **/todo** 로 시작한다.

```
@RequestMapping(method = RequestMethod.POST, value = "/todo")
public void createTodo() throws Exception {...}

@RequestMapping(method = RequestMethod.GET, value = "/todo/list")
public void getTodoList() {}
```

컨트롤러 계층

- URL 앞 공통 URL은 클래스에서 지정하고,
URL 뒤 상세 URL은 메서드에서 지정할 수 있다.

```
@RestController
@RequiredArgsConstructor
@RequestMapping("/todo")
public class TodoController {

    private final TodoService todoService;

    @RequestMapping(method = RequestMethod.POST)
    public void createTodo() throws Exception {...}

    @RequestMapping(method = RequestMethod.GET, value = "/list")
    public void getTodoList() {}

}
```

URL 뒷부분이 없다면 생략 가능

컨트롤러 계층

- 보통 편의를 위해 요청 Method 종류는 어노테이션으로 지정한다.

```
@RestController
@RequiredArgsConstructor
@RequestMapping("/todo")
public class TodoController {

    private final TodoService todoService;

    @PostMapping
    public void createTodo() throws Exception {...}

    @GetMapping("/list")
    public void getTodoList() {}

}
```

URL 뒷부분이 없다면 생략 가능

할 일 생성

- Todo Service 기능을 이용하여 할 일 생성 요청을 처리한다.

```
@PostMapping  
public void createTodo() throws Exception {  
    todoService.createTodo(content: "content", userId: 1L);  
}
```

할 일 생성

- 할 일 데이터를 생성하려면 content, user 데이터를 받아야 한다.
- Request body 데이터는 보통 json 형식으로 들어오며,
메서드 파라미터로 받을 수 있다.

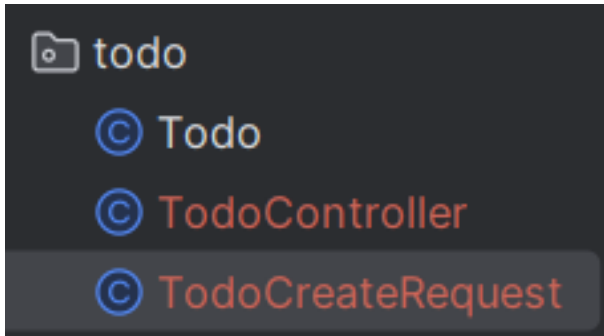
할 일 생성

- **@RequestBody**를 사용하여 파라미터로 들어오는 json 데이터를 자바 객체로 변환하여 받을 수 있다.
- 데이터를 받는 자바 객체를 **DTO**(Data Transfer Object) 라고 한다.

```
@PostMapping  
public void createTodo(@RequestBody TodoCreateRequest request) throws Exception {  
    todoService.createTodo(request.getContent(), request.getUserId());  
}
```

할 일 생성 - DTO 적용

- todo 패키지 아래 **TodoCreateRequest** 클래스 생성



```
@Getter  
public class TodoCreateRequest {  
    private String content;  
    private Long userId;  
}
```

할 일 생성 - DTO 적용

- DTO로 content, user id 받기

```
@PostMapping  
public void createTodo(@RequestBody TodoCreateRequest request) throws Exception {  
    todoService.createTodo(request.getContent(), request.getUserId());  
}
```

cf)서비스 계층에 DTO 객체를 통으로 넘기고,
DTO에서 데이터를 꺼내는 과정은 서비스 계층에서 수행하도록 구현할 수도 있다.
데이터를 꺼내는 대신 DTO 클래스에 Todo entity 객체로의 변환 메서드를 만들 수도 있다.

할 일 생성 - 응답 생성

- 컨트롤러 계층은 클라이언트의 요청을 처리한 뒤,
처리 결과를 클라이언트에게 알려주어야 한다.
- HTTP 프로토콜에 정의된 **상태코드**를 사용하여 결과를 알려주자.

프로토콜과 HTTP - 복습

대표적인 상태 코드

- 200 → 처리 성공 (ok)
- 201 → 데이터 생성 성공 (created)
- 400 → 클라이언트 요청 오류 (bad request)
- 404 → 요청 데이터 없음 (not found)
- 500 → 서버 에러 (internal server error)

할 일 생성 - 응답 생성

- 스프링이 제공하는 **ResponseEntity**를 통해 http 응답을 만들어서 반환한다.

```
@PostMapping
public ResponseEntity<Void> createTodo(@RequestBody TodoCreateRequest request) throws Exception {
    todoService.createTodo(request.getContent(), request.getUserId());
    return ResponseEntity.created(URI.create("/todo/")).build();
}
```



제네릭 타입은 Response Body로 반환할 객체의 타입을 지정한다.
데이터 생성시에는 Response Body가 없으므로 Void를 지정한다.

할 일 생성 - 응답 생성

- 스프링이 제공하는 **ResponseEntity**를 통해 http 응답을 만들어서 반환한다.

```
@PostMapping
public ResponseEntity<Void> createTodo(@RequestBody TodoCreateRequest request) throws Exception {
    todoService.createTodo(request.getContent(), request.getUserId());
    return ResponseEntity.created(URI.create("/todo/")).build();
}
```



201 created 상태코드를 갖는 응답을 생성한다.
201 응답은 생성된 데이터의 위치(path)를 함께 응답하는 것이 일반적이다.

할 일 생성 - 응답 생성

- 생성된 데이터의 위치는 /todo/{todo_id} 형태로 나타낼 수 있다.
- TodoService 에서 생성된 todo 엔티티의 id를 받아오자.

```
@Transactional
public Long createTodo(String content, Long userId) throws Exception {
    User user = userRepository.findById(userId);

    if (user == null) {
        throw new Exception("존재하지 않는 유저 ID 입니다.");
    }

    Todo todo = new Todo(content, user);
    todoRepository.save(todo);
    return todo.getId();
}
```

할 일 생성 - 응답 생성

- 생성된 데이터의 위치는 /todo/{todo_id} 형태로 나타낼 수 있다.
- TodoService 에서 생성된 todo 엔티티의 id를 받아오자.

```
@PostMapping
public ResponseEntity<Void> createTodo(@RequestBody TodoCreateRequest request) throws Exception {
    Long todoId = todoService.createTodo(request.getContent(), request.getUserId());
    return ResponseEntity.created(URI.create("/todo/" + todoId)).build();
}
```

컨트롤러 테스트

- 이제 어플리케이션을 실행하고,
API 요청을 보냈을 때 제대로 응답이 오는지 테스트해보자.
- API를 테스트하는 도구로는 **postman**을 사용한다.

컨트롤러 테스트

- <https://www.postman.com/>
- 테스트 요청을 보내기 위해 **agent 프로그램**을 설치한다.
(좌측 하단 다운로드 버튼 클릭, web에서 사용하는 경우에 필요)

Download Desktop App ↗

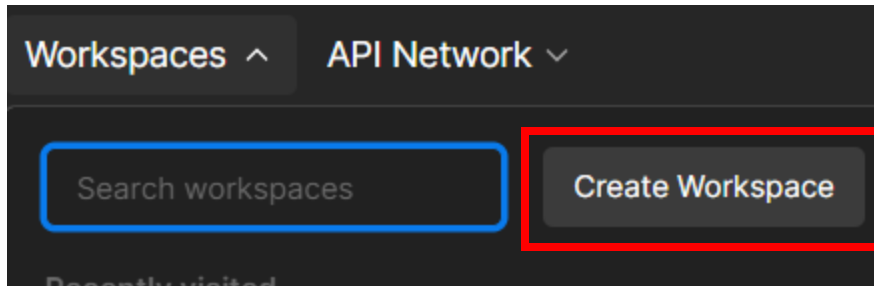
Postman desktop agent

If you are using the Postman web client, you will need to also download the Postman desktop agent. The Postman agent overcomes the Cross-Origin Resource Sharing (CORS) limitations of browsers, and facilitates API request sending from your browser version of Postman. [Read the blog post.](#)

Download Postman Agent

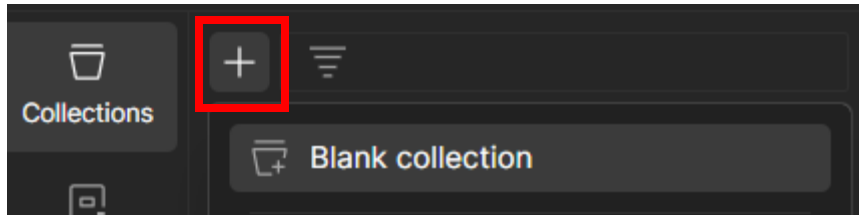
컨트롤러 테스트

- 로그인한 뒤 workspaces 를 클릭하여 새 워크스페이스를 만든다.

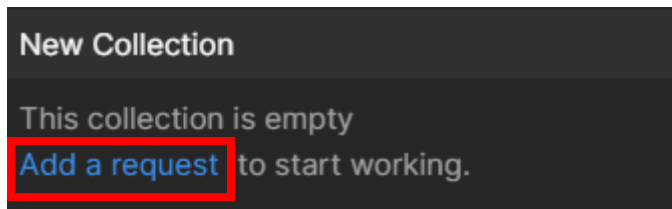


컨트롤러 테스트

- Collections 탭에서 + 버튼을 눌러 빈 컬렉션을 생성한다.

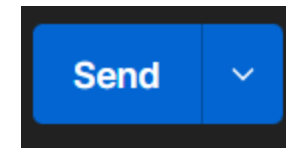
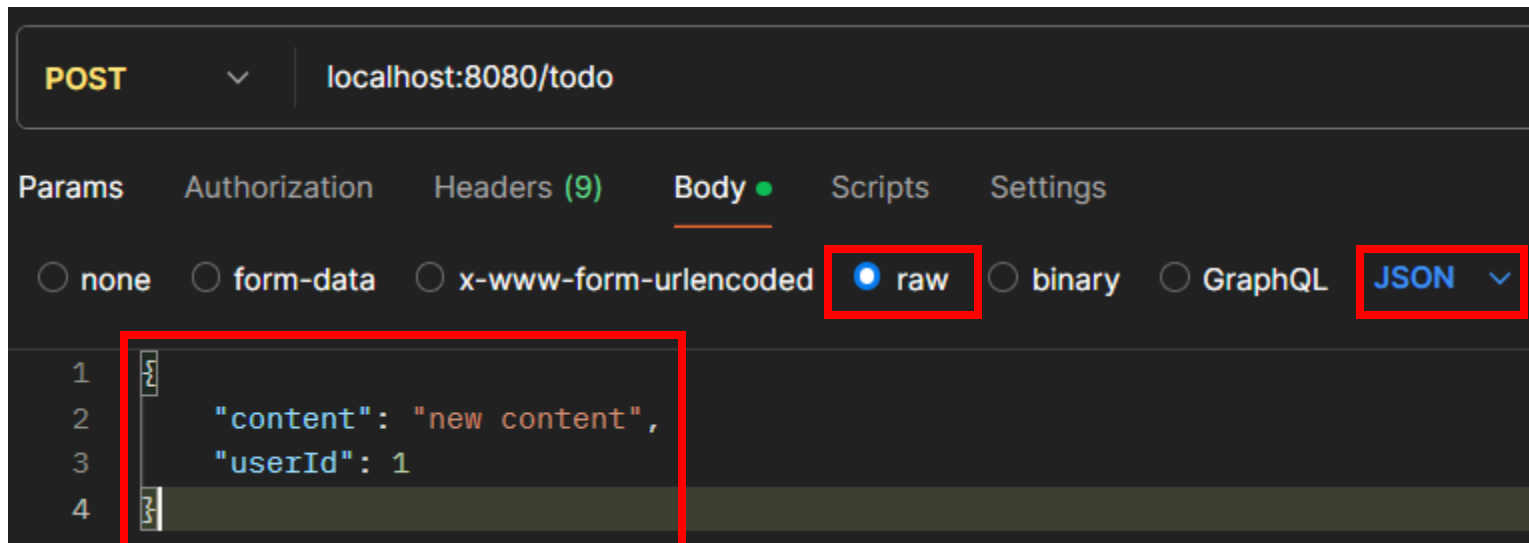


- Add a request 링크를 클릭하여 요청을 생성한다.



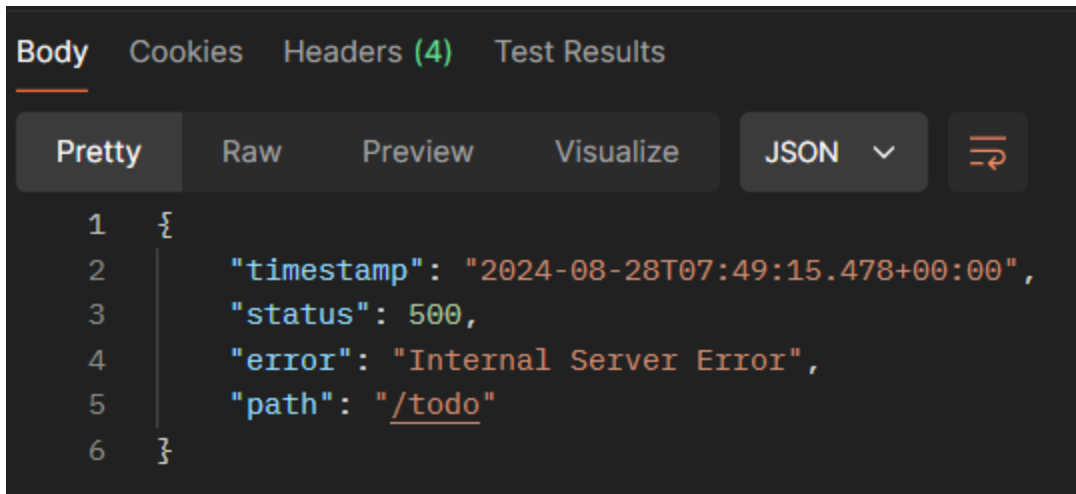
컨트롤러 테스트

- Todo 스프링 어플리케이션을 실행한 뒤,
다음과 같이 할 일 생성 요청을 작성하고, send 버튼을 클릭한다.
- **Send 버튼이 활성화되지 않았다면 agent 프로그램을 실행하자.**



컨트롤러 테스트

- 서버에서 에러가 발생하여 **500 상태코드**의 응답이 돌아온다.



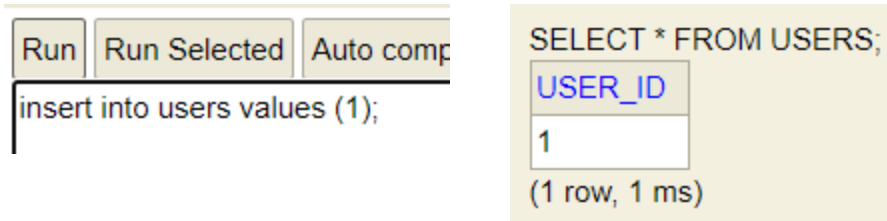
The screenshot shows a REST client interface with tabs for Body, Cookies, Headers (4), and Test Results. The Body tab is selected, and the response is displayed in a 'Pretty' JSON format. The JSON object contains a timestamp, a status of 500, an error message 'Internal Server Error', and a path of '/todo'.

```
1  {
2    "timestamp": "2024-08-28T07:49:15.478+00:00",
3    "status": 500,
4    "error": "Internal Server Error",
5    "path": "/todo"
6  }
```

```
java.lang.Exception Create breakpoint : 존재하지 않는 유저 ID 입니다.
    at com.example.todoapi.todo.TODOService.createTodo(TODOService.java:24)
```

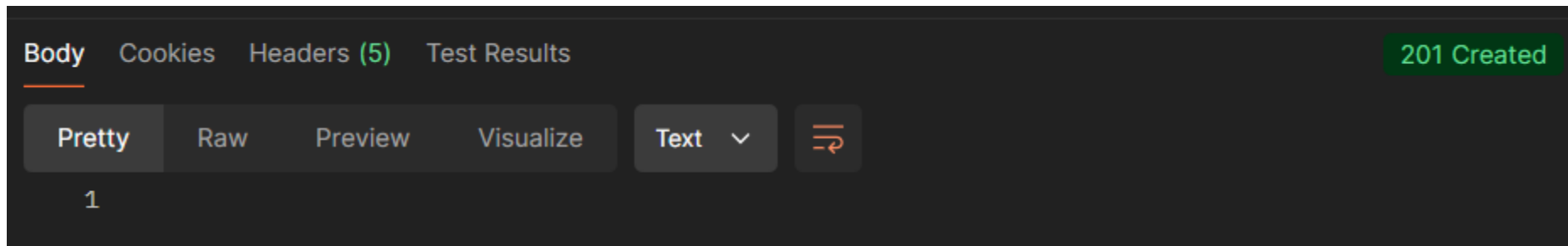
컨트롤러 테스트

- H2 콘솔에서 직접 유저를 추가하고 다시 실행하자.
(아래 그림의 유저 데이터 추가 SQL은 자신의 유저 엔티티 설계에 맞춰 수정하자)



컨트롤러 테스트

- H2 콘솔에서 직접 유저를 추가하고 다시 실행하자.



```
SELECT * FROM TODO;
```

TODO_IS_CHECK	TODO_ID	USER_ID	TODO_CONTENT
0	1	1	new content

(1 row, 4 ms)

할 일 조회

```
@GetMapping("/list")
public ResponseEntity<List<Todo>> getTodoList(@RequestBody Long userId) throws Exception {
    List<Todo> todos = todoService.readTodos(userId);
    return ResponseEntity.ok(todos);
}
```

할 일 조회

```
@GetMapping("/list")
public ResponseEntity<List<Todo>> getTodoList(@RequestBody Long userId) throws Exception {
    List<Todo> todos = todoService.readTodos(userId);
    return ResponseEntity.ok(todos);
}
```



제네릭 타입은 Response Body로 반환할 객체의 타입을 지정한다.
여러 개의 Todo List가 반환되므로 List<Todo>를 지정한다.

지금은 편의를 위해 엔티티 클래스를 사용했지만,
Response에도 DTO를 사용하는 것이 일반적이다.
(List<TodoResponse>와 같이 DTO를 리스트로 반환하면 된다.)

할 일 조회

```
@GetMapping("/list")
public ResponseEntity<List<Todo>> getTodoList(@RequestBody Long userId) throws Exception {
    List<Todo> todos = todoService.readTodos(userId);
    return ResponseEntity.ok(todos);
}
```



조회시 경우 성공하면 200 OK 를 응답한다.

할 일 조회

```
@GetMapping("/list")
public ResponseEntity<List<Todo>> getTodoList(@RequestBody Long userId) throws Exception {
    List<Todo> todos = todoService.readTodos(userId);
    return ResponseEntity.ok(todos);
}
```

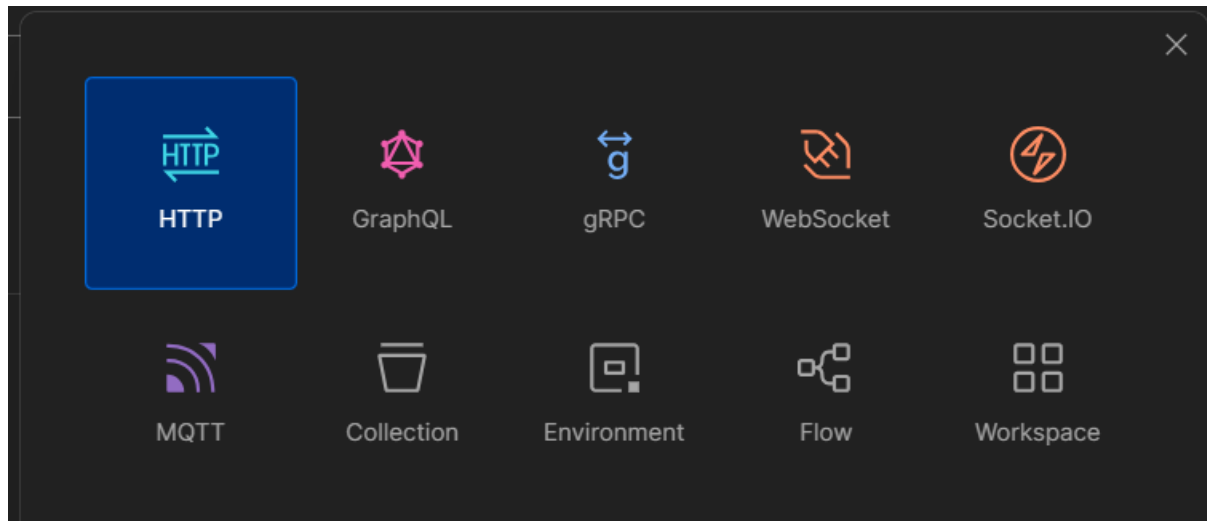
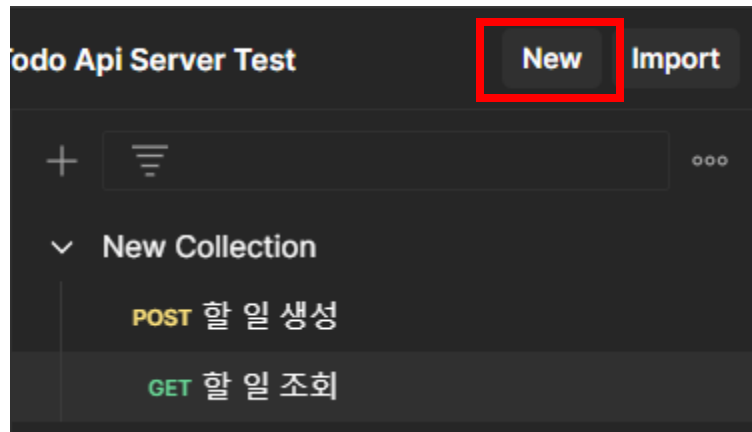


간단하게 작성하기 위해
DTO 클래스 대신 참조타입을 사용했다.
참조타입은 response body에 있는 데이터를 그대로 받아들인다.

(사실 로그인한 유저 정보는 body에 직접 넣지 않는다.
관심이 있다면 Spring Security와 세션, JWT를 공부해보자.)

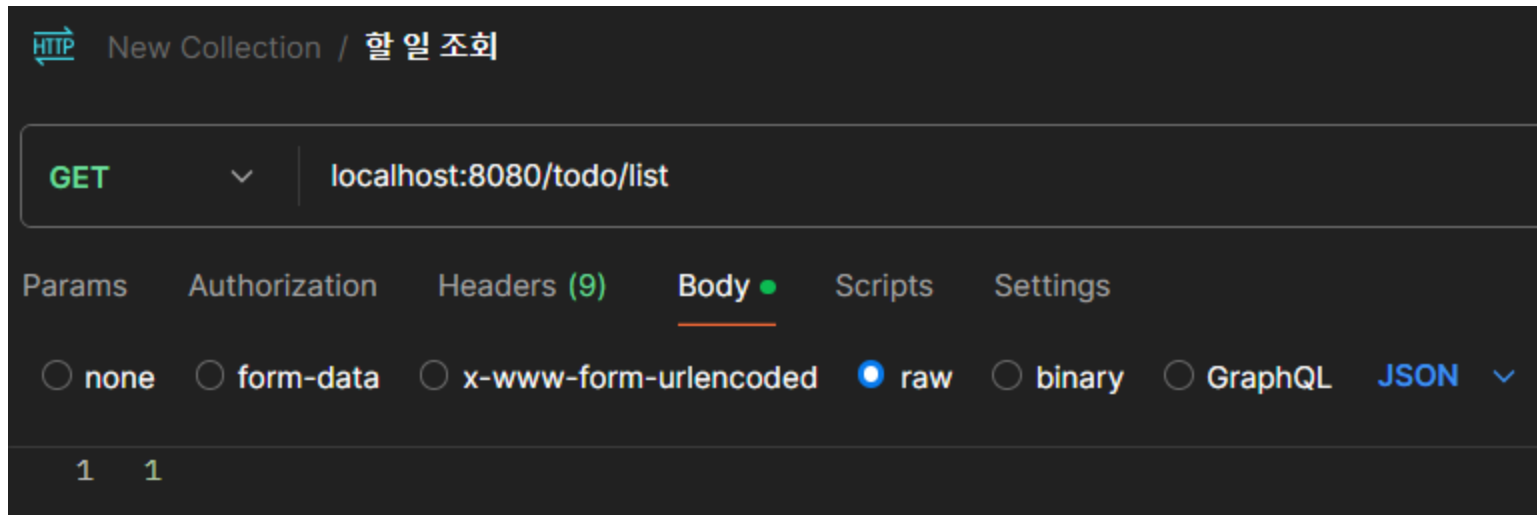
할 일 조회 테스트

- 새로운 HTTP 요청을 컬렉션에 추가한다.



할 일 조회 테스트

- 유저와 할 일 데이터를 미리 생성해둔다.
- 그림과 같이 작성하고 ctrl + s 를 눌러 저장한다.



할 일 조회 테스트

- 실행 결과

```
[
  {
    "id": 1,
    "content": "new content",
    "user": {},
    "checked": false
  },
  {
    "id": 2,
    "content": "new conten2",
    "user": {},
    "checked": false
  }
]
```

할 일 삭제

- Path variable은 url에 {변수명} 형태로 명시한다.
- 변수명과 같은 이름의 매개변수 앞에 @PathVariable을 사용하여 path parameter 값을 매개변수로 받을 수 있다.

```
@DeleteMapping("/{todoId}")  
public ResponseEntity<Void> deleteTodo(@PathVariable Long todoId) {  
    todoService.deleteTodo(todoId);  
    return ResponseEntity.noContent().build();  
}
```

할 일 삭제

- 삭제의 경우, 보통 **204 no content** 상태코드를 응답한다.

```
@DeleteMapping("/{todoId}")
public ResponseEntity<Void> deleteTodo(@PathVariable Long todoId) {
    todoService.deleteTodo(todoId);
    return ResponseEntity.noContent().build();
}
```

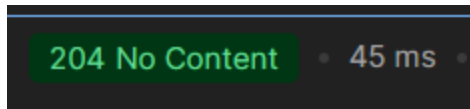
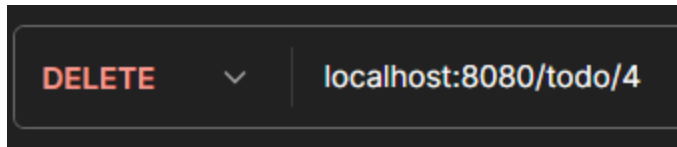
할 일 삭제 테스트

- 미리 유저와 삭제할 데이터를 생성해둔다.

```
{  
  {  
    "id": 2,  
    "content": "new conten2",  
    "user": {},  
    "checked": false  
  },  
  {  
    "id": 3,  
    "content": "new conten2",  
    "user": {},  
    "checked": false  
  },  
  {  
    "id": 4,  
    "content": "new content3",  
    "user": {},  
    "checked": false  
  }  
}
```


할 일 삭제 테스트

- 데이터를 삭제하고, 다시 조회하여 삭제된 것을 확인한다.



할 일 수정

- **@RequestBody, @PathVariable** 을 모두 사용한다.
(수정할 content는 간단하게 String으로 받았다. 구현에 따라 DTO로 받아도 된다.)

```
@PatchMapping("/{todoId}")
public ResponseEntity<Void> updateTodoContent(@PathVariable Long todoId, @RequestBody String newContent) {
    todoService.updateContent(todoId, newContent);
    return ResponseEntity.ok().build();
}
```

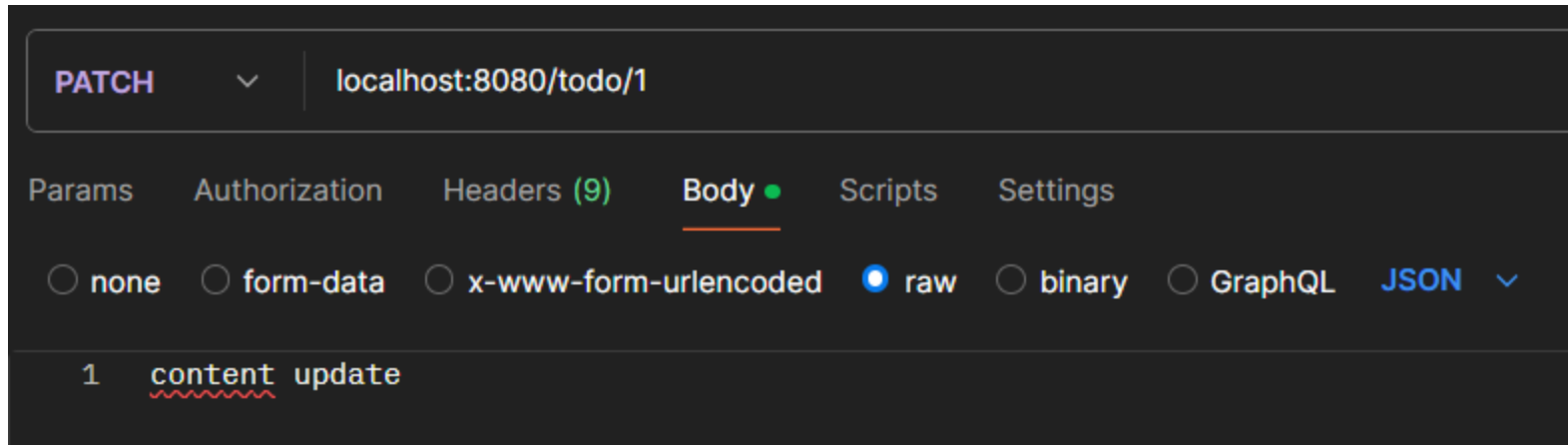
할 일 수정 테스트

- 유저와 수정할 데이터를 미리 생성한다.

```
[
  {
    "id": 1,
    "content": "new content3",
    "user": {},
    "checked": false
  },
  {
    "id": 2,
    "content": "new content3",
    "user": {},
    "checked": false
  }
]
```

할 일 수정 테스트

- 할 일 수정 요청을 보내고, 수정이 된 것을 확인한다.
(수정할 content를 String으로 받으므로 request body에 문자열로 넘긴다.)



```
[
  {
    "id": 1,
    "content": "content update",
    "user": {},
    "checked": false
  },
  {
    "id": 2,
    "content": "new content3",
    "user": {},
    "checked": false
  }
]
```

프로젝트 – 과제

- 모든 도메인에 대해 컨트롤러 계층 작성하기
- Postman으로 API 테스트 해보기

프로젝트 명세

Todo mate API 서버 클론 코딩

<주요 기능>

- 유저 회원가입, 로그인
- 로그인한 유저의 할 일 생성 / 조회 / 수정 / 삭제
- 로그인한 유저의 할 일 체크 / 체크 해제
- 다른 유저에게 친구 요청 / 요청 수락 / 친구 조회 / 친구 삭제
- 친구 유저의 할 일 조회